EXPERIMENT 1

Object Detection and Recognition

Dataset Engineering · YOLOv8 Training · Edge Deployment

GROUP 11

Hua Yingjie

Student ID 24020036025

YOLOv8 · Jetson · ROS2
Experimental Report

Abstract

This report presents an end-to-end desktop object detection system designed for deployment on an NVIDIA Jetson edge device. The system detects three classes—mouse, keyboard, and cup—and covers data acquisition, automatic pre-annotation, manual review, dataset auditing, YOLOv8 transfer learning, independent evaluation, real-time camera inference, and a ROS2-compatible output interface. The integrated dataset contains approximately 650 images from self-collected scenes, supplementary online samples, and negative backgrounds. A fully audited core subset contains 225 images and 687 positive bounding boxes; 15 orphan annotation files were identified and isolated during quality control. YOLOv8n was trained for 100 epochs on an Apple Silicon Mac using the PyTorch MPS backend. Validation achieved a precision of 0.806, recall of 0.894, mAP@0.5 of 0.888, and mAP@0.5:0.95 of 0.814. The final Jetson video contains 300 frames at 1280×720 over 20 s and consistently displays 15 FPS, exceeding the required 5 FPS while detecting all three target classes.

Keywords: object detection; YOLOv8; Jetson; automatic pre-annotation; dataset quality audit; edge deployment; ROS2

Contents

1 Objectives and Requirements

Desktop object detection is a fundamental perception task in robotic manipulation, visual servoing, and human-robot interaction. Unlike an offline benchmark, this experiment requires both model training and deployment on a Jetson platform with a live camera and a processing speed of at least 5 FPS. The work therefore considers data quality, detection accuracy, inference latency, and interface extensibility as one engineering system.

The objectives are to build a three-class dataset, accelerate annotation through human-reviewed pre-annotations, train and evaluate YOLOv8n, deploy the model on Jetson, visualize class labels, boxes and confidence values, and provide structured detection output for ROS2 integration.

Table 1 Course requirements and corresponding evidence

Requirement	Completion and evidence
At least two desktop-object classes	Three classes: Mouse, Keyboard, and Cup
Self-collection, annotation, and training	Multi-source acquisition, pre-annotation, manual review, and YOLOv8 training completed
Run recognition on Jetson	Live camera inference completed; see Section 7
Show class, box, and confidence	All three items are overlaid in the recorded output
Publish detection results through ROS2	Publisher and message schema completed; payload validated offline on Mac, while live topic echo remains an on-device integration item
Test 20 objects with at least 80% accuracy	A controlled procedure and record sheet are provided for on-site acceptance
At least 5 FPS on Jetson	Recorded interface displays a stable 15 FPS
Save results and representative errors	Final video and a low-confidence false-positive example are retained

2 System Architecture

The workflow consists of data engineering, model training, edge deployment, and error-driven feedback. The Mac performs data cleaning, splitting, training, and evaluation. The Jetson performs camera capture, inference, visualization, logging, and message publication. Misclassified, missed, or low-confidence frames can be returned to the labeling stage for iterative improvement.

3 Experimental Environment

Training was performed on an Apple Silicon Mac using the PyTorch MPS backend. Deployment used an NVIDIA Jetson edge-computing board connected to a camera. The software stack comprised Python, Ultralytics YOLOv8, PyTorch, OpenCV, the CUDA/TensorRT runtime, and a ROS2 interface.

4 Dataset Construction and Quality Control

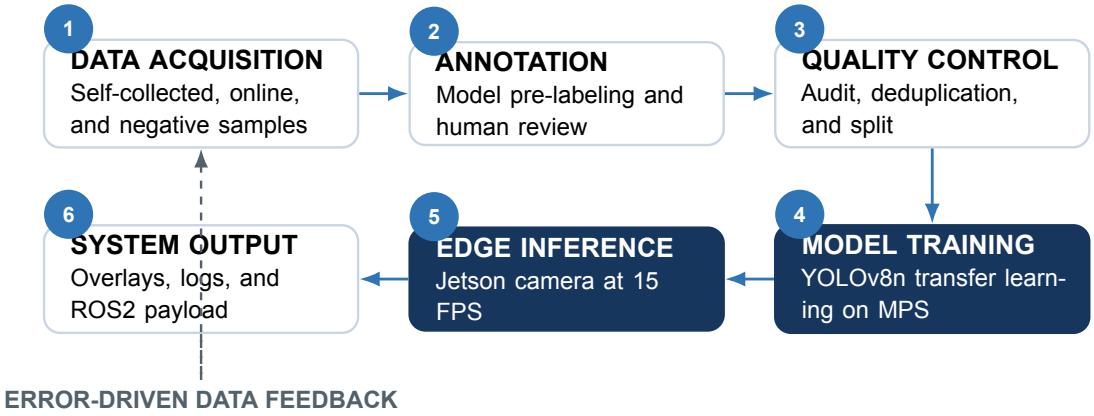


Figure 1 Six-stage detection pipeline with deployment feedback to dataset refinement

Table 2 Hardware and software environment

Item	Configuration and purpose
Training platform	Apple Silicon Mac with MPS acceleration
Deployment platform	NVIDIA Jetson with a live camera
Detection framework	Ultralytics YOLOv8 and PyTorch
Image processing	OpenCV capture, drawing, display, and recording
Communication	ROS2-compatible JSON publisher
Target classes	Mouse, Keyboard, Cup

4.1 Multi-source data design

The dataset uses self-collected images as the primary source and online images as supplementary material. Self-collected data matches the deployment domain and covers different desks, lighting conditions, distances, scales, poses, and occlusion levels. Online samples improve diversity in object appearance, color, shape, and background. Their licenses, class definitions, and annotations must be reviewed before use.

The integrated collection contains approximately 650 images, including positive scenes and negative samples. Negative scenes contain empty desks, cables, monitor borders, packages, and other visually similar distractors. They help suppress background false positives. The fully audited core subset contains 225 images and 687 positive boxes. A negative image contributes to the image count but contains no positive box.

4.2 Automatic pre-annotation and manual review

A general-purpose pretrained detector first generated candidate boxes. These candidates were converted to YOLO format and then reviewed image by image. Manual review corrected box boundaries and class identifiers, restored missed objects, and removed false annotations. This human-in-the-loop process changes labeling from drawing every box from scratch to validating and correcting candidates, improving efficiency without treating machine predictions as ground truth.

4.3 Positive, negative, and hard cases

Positive samples include isolated objects, multi-object scenes, partial occlusions, and scale changes. Hard cases cover low illumination, reflection, motion blur, heavy occlusion, small objects, clutter, and extreme viewpoints. Deployment errors should be categorized and returned to the training set only after manual annotation. This produces a traceable error-driven data loop instead of relying only on threshold adjustment.

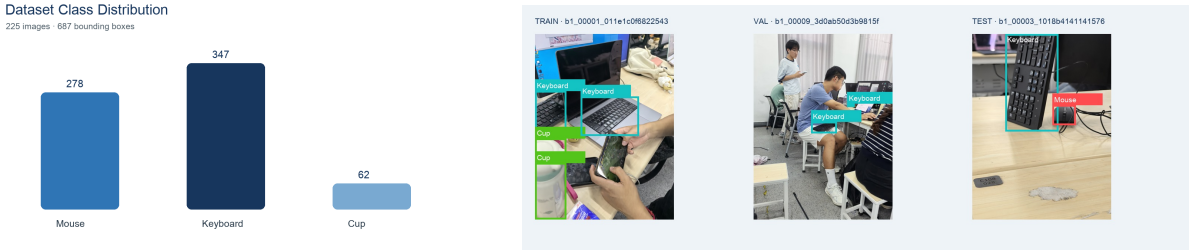
4.4 Dataset audit and split

The audit checked image-label correspondence, legal class identifiers, normalized coordinates within $[0, 1]$, positive box dimensions, and orphan files. Fifteen labels without matching images were isolated. The audited subset was divided with random seed 42 into 180 training images, 22 validation images, and 23 test images. Consecutive frames and near-duplicate samples were grouped before splitting to reduce leakage.

Table 3 Bounding-box distribution in the audited subset

Class	Boxes	Share (%)
Mouse	278	40.47
Keyboard	347	50.51
Cup	62	9.02
Total	**687**	**100.00**

Cup accounts for only 9.02% of the positive boxes and is therefore a long-tail class. Per-class recall and AP are important in addition to overall mAP.



(a) Bounding-box distribution **(b)** YOLO labels rendered on audited images

Figure 2 Dataset statistics and annotation evidence generated from the audited subset

5 YOLOv8 Model Training

YOLOv8n was selected as the edge baseline because of its small parameter count and low latency. Training started from general pretrained weights with an input size of 640×640, batch size 8, and 100 epochs on MPS. The objective combines box regression, classification, and distribution focal loss:

$$\mathcal{L} = \lambda_{box}\mathcal{L}_{box} + \lambda_{cls}\mathcal{L}_{cls} + \lambda_{dfl}\mathcal{L}_{dfl}. \tag{1}$$

Brightness and saturation perturbation, scale, translation, horizontal flipping, and Mosaic augmentation improve generalization when their parameters are validated rather than maximized. Strong unrealistic rotations are avoided for keyboards with directional layouts.

For evaluation,

$$P = \frac{TP}{TP + FP}, \qquad R = \frac{TP}{TP + FN}. \tag{2}$$

AP is the area under the precision-recall curve, while mAP averages AP across classes. Both mAP@0.5 and mAP@0.5:0.95 are reported to avoid overestimating localization quality at a single IoU threshold.

YOLOv8s may be evaluated under the same split, image size, epochs, and test procedure. It should replace YOLOv8n only if its accuracy gain justifies the additional latency and it remains above 5 FPS on Jetson. This is a planned controlled comparison, not a claimed completed result.

6 Training and Validation Results

Table 4 YOLOv8n validation results

Metric	Value	Assessment	Interpretation
Precision	0.806	High	Relatively few false positives
Recall	0.894	High	Relatively few missed targets
mAP@0.5	0.888	Good	Strong performance at IoU 0.5
mAP@0.5:0.95	0.814	Good	Robust localization under stricter IoU

The recorded Mac timings were 1.3 ms for preprocessing, 27.7 ms for network inference, and 10.3 ms for postprocessing, giving

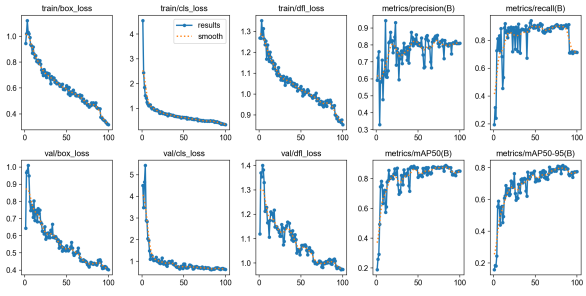
$$\text{FPS} \approx \frac{1000}{1.3 + 27.7 + 10.3} = 25.45. \tag{3}$$

This is a host-side value and does not replace end-to-end Jetson measurement, which includes capture, rendering, and scheduling overhead.

(a) Normalized confusion matrix **(b)** Validation labels **(c)** Validation predictions

Figure 3 Validation and error-analysis evidence retained by the training pipeline

Mouse and Keyboard have more training instances and more stable outlines. Cup is less frequent and varies substantially because of transparency, reflection, handles, and background color. Confidence thresholds therefore require class-aware review: lowering the threshold improves recall but introduces more false positives, whereas raising it can suppress small or occluded targets.



(a) Training commands recorded on the Mac **(b)** Metrics over 100 epochs

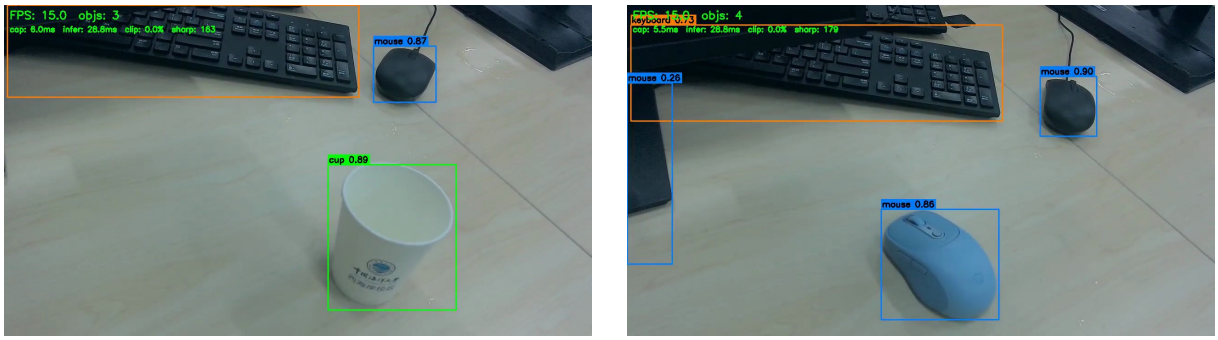
Figure 4 Training execution and convergence evidence

7 Jetson Deployment and Runtime Results

The trained `best.pt` model was transferred to Jetson. OpenCV captures each frame, YOLOv8 returns class identifiers, confidence scores, and bounding boxes, and the program draws the results while measuring FPS. The program can save video and CSV logs and can publish the same structured output for downstream perception or control nodes.

TensorRT FP16 export is a reproducible optimization path that can reduce compute and memory use. A valid comparison must keep image resolution, confidence threshold, camera input, and power mode fixed and report pure inference latency, end-to-end latency, mean and minimum FPS, and memory usage. TensorRT acceleration is not presented here as a completed measurement.

The final video is 1280×720, 300 frames, and 20 s at 15 FPS. Twelve uniformly spaced frames all displayed 15.0 FPS. Figure 5 shows simultaneous detection of Keyboard, Mouse, and Cup, including a Cup confidence of 0.89.



(a) Three classes detected; Cup confidence 0.89 **(b)** Keyboard and multiple mice detected

Figure 5 Real-time Jetson camera detections extracted from the final video

Table 5 Measured results from the final Jetson video

Test item	Measured result
Video specification	1280×720, 300 frames
Duration and frame rate	20 s, 15 FPS
Representative inference time	28.8 ms
Representative image-processing time	6.0 ms
Detected classes	Keyboard, Mouse, Cup
Real-time requirement	15 FPS $>$ 5 FPS; requirement satisfied

7.1 ROS2 interface and offline verification

The deployment program implements an `rclpy` node named `desk_object_detector`. It publishes JSON through `std_msgs/String` on `/desk_object_detections`. Each payload contains the frame index, timestamp, FPS, class identifier, class name, confidence, and pixel coordinates $[x_1, y_1, x_2, y_2]$.

ROS2, `colcon`, and `rclpy` were not installed on the Mac. Consequently, the evidence below is an actual serialization, deserialization, and schema check of the same payload, not a fabricated `ros2 topic echo`. Live topic echo remains to be verified in the Jetson ROS2 environment.

For the required 20-object acceptance test, at least 20 physical instances should cover the three classes, viewpoints, distances, and occlusion levels. Each correct detection, missed detection, and false detection must be recorded. The acceptance accuracy is the number of correctly recognized objects divided by the total number tested. This result must be reported separately from mAP.

Figure 6 Offline validation of the ROS2 message payload on Mac (not a live topic echo)

8 Error Analysis and Improvements

A low-confidence Mouse candidate with confidence 0.26 appears near the left border of one recorded frame. The result illustrates the precision-recall trade-off: a low threshold preserves difficult targets but may accept background structures. Adding desk edges, chassis edges, cables, and dark elongated objects as negative samples can reduce this error.

Domain shift also exists between training images and Jetson video because of focal length, exposure, motion blur, and viewpoint. Cup is especially vulnerable under reflections, low light, and background-color similarity. Error frames should be categorized as low light, occlusion, small object, clutter, or motion blur, manually reviewed, and added to the next training round. A controlled ablation can then compare the baseline, augmentation, negative samples, and hard-case feedback while keeping the split and training settings fixed.

9 Engineering Contributions

First, automatic pre-annotation followed by manual review provides an efficient and reliable human-in-the-loop dataset workflow. Second, the project treats dataset construction as quality engineering by checking orphan labels, invalid classes, out-of-range coordinates, imbalance, and near duplicates. Third, the detector is evaluated as an edge system rather than only an offline model: camera input, overlays, frame rate, saved evidence, and a ROS2-compatible output are integrated. Finally, deployment errors are returned to the dataset, forming a continuous improvement loop that is more informative than simply increasing the epoch count.

10 Project Archive and Reproducibility

Code, dataset configuration, audited labels, the trained model, training results, and the report are organized in the personal repository Gabriel6002/-1. Large image collections and the final video are intentionally excluded from GitHub. The repository uses one clear location for each artifact category: `scripts/`, `data/`, `models/`, `results/`, `report/`, and `docs/`.

11 Conclusion

This experiment completed an end-to-end desktop object detector from multi-source dataset construction to real-time Jetson execution. The integrated dataset contains approximately 650 images, including self-collected positives, supplementary online samples, and negative backgrounds. The audited core contains 225 images and 687 positive boxes, and quality control prevented 15 orphan labels from entering training. YOLOv8n achieved precision 0.806, recall 0.894, mAP@0.5 0.888, and mAP@0.5:0.95 0.814. The final video demonstrates real-time detection of Keyboard, Mouse, and Cup at a stable displayed rate of 15 FPS,

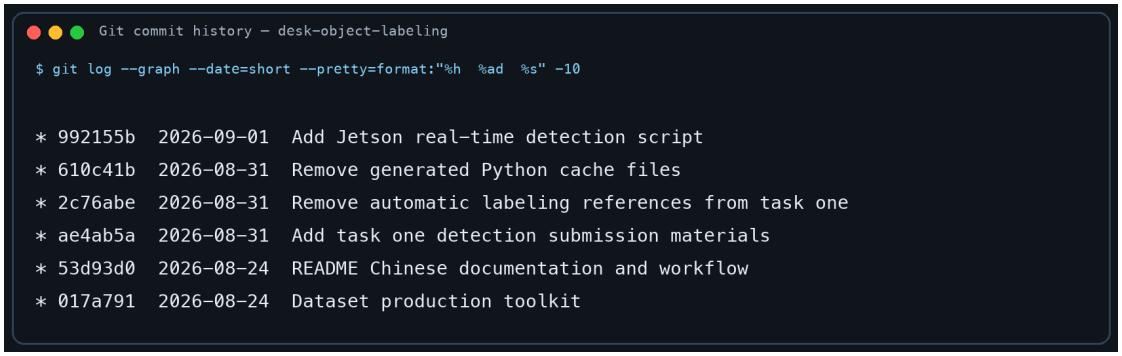


Figure 7 Incremental Git commit history generated from the local repository log

satisfying the course requirement.

The work shows that a complete object-detection experiment requires dataset quality control, independent evaluation, hardware deployment, latency measurement, reproducible archiving, and error feedback. Future work should expand Cup and negative samples and complete live ROS2 topic verification on Jetson.

References

[1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLO," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. CVPR*, 2016, pp. 779–788.

[3] NVIDIA, "NVIDIA TensorRT Developer Guide." [Online]. Available: https://docs.nvidia.com/deeplearning/tensorrt/

[4] Open Robotics, "ROS 2 Documentation." [Online]. Available: https://docs.ros.org/